

MAIE 6000C: Engineering AI Products: From Prototype to Production

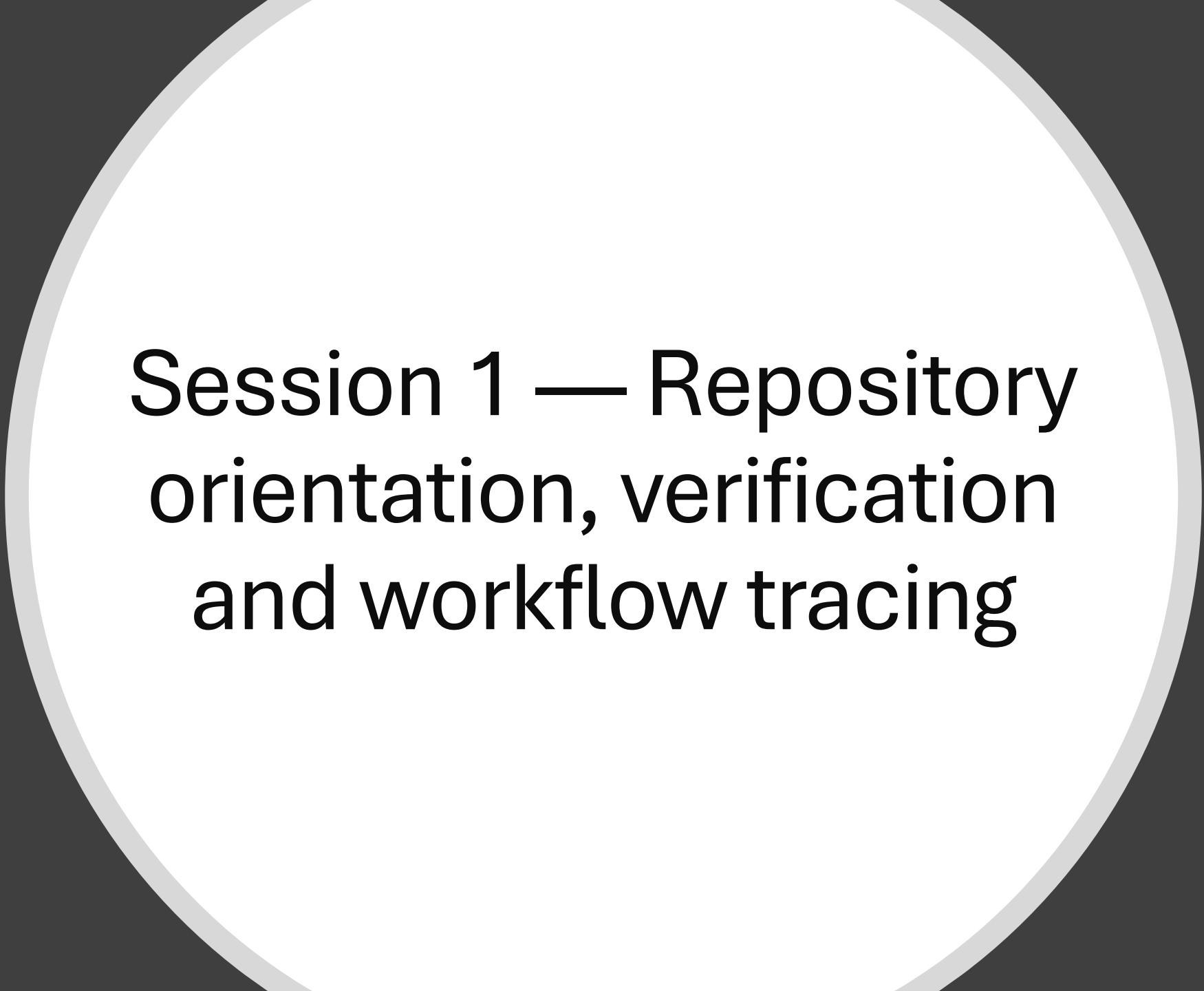
Week 3 – Lab

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lfteris NTAFL0S, PhD

The Hong Kong University of Science and Technology



Session 1 — Repository orientation, verification and workflow tracing

Welcome,
lab
distinction,
and "start
the stack
now"

Week 3 Lab — Session 1: establish a reliable baseline before you change anything

Do this now, in the background: docker compose up --build -d from your repository root

This lab *prepares* you for the W03 Individual Readiness Lab. It does not complete it. The assessed change is done individually in your own repository, tagged w03-readiness, submitted via Canvas.

Session goals and run of show

By the end of Session 1 you will have:

1. Confirmed repo identity and Git baseline
2. Mapped the repository
3. Confirmed reproducible startup
4. Seen liveness vs readiness demonstrated
5. Smoke-tested health endpoints
6. Traced one case end-to-end: API → DB → job → worker → AI → result → log
7. Found and run the project's tests
8. Written a baseline evidence note



Ground rules + DEMO: Git baseline

```
git status  
git branch --show-current  
git log --oneline -5  
git remote -v
```

Do NOT:

- Edit directly inside a Docker container
- Make changes you cannot explain
- Combine several unrelated changes
- `run git reset --hard` or `docker compose down -v` casually
- Treat your lab notes as the submission

Activity 1: Your Git baseline + repository map (6min)

Part A (3 min): run the four commands of the previous slide and record: clean tree? branch? remote points to *your* repo? latest commit message? anything unexpected?

Part B (3 min): open the repo and write your own map of where each responsibility lives. Actual layout of this starter:

compose.yaml, Dockerfile, Makefile	→ startup and commands
services/api/app/main.py	→ public API (FastAPI)
services/ai/app/main.py	→ internal AI service (/trriage)
services/worker/app/main.py	→ job poller
services/common/{models,schemas,db,config}.py	→ shared DB models, Pydantic schemas, settings
alembic/versions/	→ migrations
tests/unit integration smoke	→ verification
docs/architecture.md, operations.md, adrs/	→ living documents
.github/workflows/ci.yml	→ CI

- Where does a request enter?
- Where is state stored?
- What triggers background work?
- Where is the AI called?

ACTIVITY 2: Confirm reproducible startup (6min)

```
docker compose config      # is the compose file valid? which ports?
docker compose ps          # which services are running / healthy?
docker compose logs --tail=30 # any startup errors?
```

Record:

- services running (db, ai, api, worker)
 - any failing service
 - API host port from compose.yaml/.env (API_HOST_PORT, default 8000; AI 8100)
 - which health endpoint should respond.
-

DEMO: Liveness vs readiness

```
curl -i http://localhost:8000/health/live # → 200 {"service":"api","status":"ok"}  
curl -i http://localhost:8000/health/ready # → 200 (runs SELECT 1 against Postgres)  
docker compose stop db # simulate DB outage  
curl -i http://localhost:8000/health/live # → still 200 (process alive)  
curl -i http://localhost:8000/health/ready # → 500 (dependency down)  
docker compose logs worker --tail=5 # job_processing_failed every 2 s  
docker compose start db  
curl -i http://localhost:8000/health/ready # → 200 again
```

- Check your .env if your API_HOST_PORT is 8000 or 8001
- In PowerShell use curl.exe
- In Mac use curl

ACTIVITY 3: Smoke-test the running system (4min)

via `http://localhost:<port>/docs` or `curl`:

- GET /
- GET /health/live
- GET /health/ready
- GET /metrics (look for `api_http_requests_total`)
- Also the AI service:
`http://localhost:8100/health/live`
Record:
 - status code
 - Body
 - alive?
 - ready?
- one relevant log line (`docker compose logs api --tail=10`
→ `request_completed`).

ACTIVITY 4: Trace one case end-to-end (10min)

API base URL

```
API_URL="http://localhost:8000"
```

1. Create a case

```
curl -i -X POST "$API_URL/cases" -H "Content-Type: application/json" \
--data-raw '{"title":"Cannot login to portal","description":"Password reset did not restore access to the dashboard."}'
```

Expected initially: 201 Created / case.status = "queued" / job.status = "pending" / job.attempts = 0

2. Retrieve the case

Replace <case_id> with the ID returned by the POST request

```
curl -s "$API_URL/cases/<case_id>"
```

Expected later: status = "triaged" / ai_label = "access" / ai_confidence = ...

3. Retrieve the job. Replace <job_id> with the job ID returned by the POST request

```
curl -s "$API_URL/jobs/<job_id>"
```

Expected later: status = "completed" / attempts = 1 / claimed_at = ... / completed_at = ...

4. Check persisted cases in PostgreSQL

```
docker compose exec db psql -U postgres -d maie6000c -c \
"SELECT id, status, ai_label, ai_confidence
FROM cases
ORDER BY created_at DESC
LIMIT 3;"
```

5. Check persisted jobs in PostgreSQL

```
docker compose exec db psql -U postgres -d maie6000c -c \
"SELECT id, case_id, status, attempts, error
FROM jobs
ORDER BY id DESC
LIMIT 3;"
```

6. Check asynchronous worker and AI logs

```
docker compose logs worker --tail=10
docker compose logs ai --tail=5
```

- These commands work on Mac
- Check next slide for PowerShell
- Also try the above on Swagger!

ACTIVITY 4: Trace one case end-to-end (10min)

```
# API base URL
$ApiUrl = "http://localhost:8000"

# 1. Create a case
$Body = @{
    title = "Cannot login to portal"
    description = "Password reset did not restore access to the dashboard."
} | ConvertTo-Json -Compress

$Created = Invoke-RestMethod -Uri "$ApiUrl/cases" -Method Post `
    -ContentType "application/json" -Body $Body

$Created | ConvertTo-Json -Depth 10

# Capture the returned IDs
$CaseId = $Created.case.id
$JobId = $Created.job.id

Write-Host "Case ID: $CaseId"
Write-Host "Job ID: $JobId"

# Expected initially: 201 Created / case.status = "queued" / job.status = "pending" /
job.attempts = 0

# Wait for asynchronous processing
Start-Sleep -Seconds 5
```

```
# 2. Retrieve the case
Invoke-RestMethod -Uri "$ApiUrl/cases/$CaseId" | ConvertTo-Json -Depth 10

# Expected later: status = "triaged" / ai_label = "access" / ai_confidence = ...

# 3. Retrieve the job
Invoke-RestMethod -Uri "$ApiUrl/jobs/$JobId" | ConvertTo-Json -Depth 10

# Expected later: status = "completed" / attempts = 1 / claimed_at = ...
completed_at = ...

# 4. Check persisted cases in PostgreSQL
docker compose exec db psql -U postgres -d maie6000c -c "SELECT id,
status, ai_label, ai_confidence FROM cases ORDER BY created_at DESC
LIMIT 3;"

# 5. Check persisted jobs in PostgreSQL
docker compose exec db psql -U postgres -d maie6000c -c "SELECT id,
case_id, status, attempts, error FROM jobs ORDER BY id DESC LIMIT 3;"

# 6. Check asynchronous worker and AI logs
docker compose logs worker --tail=10
docker compose logs ai --tail=5
```

- These commands work n PowerShell
- Also try the above on Swagger!

ACTIVITY 5: Find and run verification (6 min)

Where verification lives in this repo:

- tests/ (unit, integration, smoke)
- pyproject.toml [tool.pytest.ini_options]
- Makefile (make test, make smoke)
- .github/workflows/ci.yml.

ACTIVITY 5: Find and run verification (6 min)

First edit Dockerfile in your repo (replace your existing Dockerfile):

```
FROM python:3.11-slim
ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1
WORKDIR /app
COPY pyproject.toml README.md /app/
COPY services /app/services
COPY scripts /app/scripts
COPY alembic /app/alembic
COPY alembic.ini /app/alembic.ini
RUN pip install --upgrade pip && pip install -e ".[dev]"
COPY tests /app/tests
CMD ["uvicorn", "services.api.app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

ACTIVITY 5: Find and run verification (6 min)

Stage, commit, and push your changes:

```
git add .  
git commit -m "Updated Dockerfile so we can run tests from image"  
git push
```

Rebuild the image:

```
docker compose build --no-cache api
```

Then verify that the tests (smoke, unit, integration) exist inside the container:

```
docker compose run --rm --no-deps api sh -c "find /app/tests -maxdepth 3 -type f"
```

Run unit and integration tests:

```
docker compose run --rm --no-deps api pytest -q tests/unit tests/integration
```

Expected result: 4 passed

CHECKPOINT: Baseline evidence note (4 min)

Save a personal note containing:

- Repository (remote URL):
- Current branch:
- Working tree status:
- Configured API port:
- Running services:
- Health-check results (live / ready):
- One case traced (case_id, final status, ai_label):
- Verification command used:
- Result:
- One question or blocker:



SESSION 2 — Readiness planning and safe-change rehearsal

Recap, goals and boundary

Rehearse

Session 1: baseline established. Session 2: rehearse the safe-change loop, then plan your bounded change.

Leave

Leave with: a candidate bounded change · a reason · a verification plan · a Git plan · a documentation plan · blockers list.

Do not leave

Do not leave with: a copied solution · a large feature plan · a tag created before the work is done.

Exact deliverables of the W03 Individual Readiness Lab



Repository must contain: bounded change · test/verification update · submissions/week03/README.md (create the folder; start from templates/submissions/week03-README-template.md) · tag w03-readiness on GitHub.



Canvas text entry: Repository · Checkpoint tag: w03-readiness · Commit SHA (`git rev-parse w03-readiness^{commit}`) · What I changed · How I verified it · AI Use Statement.



Recommended path: branch → PR → merge to main → tag on main

ACTIVITY 6: Bounded or too large? (5 min)

Vote for each: **Bounded (B)** or **Too large (T)**:

1. Add a test that GET /cases/unknown-id returns 404
2. Add JWT authentication to all endpoints
3. Make /health/ready return 503 with a message when the DB is unreachable
4. Replace the keyword AI service with an LLM API
5. Add an optional source field to Case, persist it (migration), return it, test it
6. Build a React dashboard for cases
7. Fix docs/operations.md: document pip install -e .[dev] and the Python 3.11 requirement before pytest
8. Add GET /jobs?status=pending list endpoint with a test
9. Add a retry mechanism to the worker with exponential backoff, plus a priority field, plus fixing the logs

DEMO: The full safe-change loop

Watch the demo in Powershell:

1. Run the commands to check git: `git status | git branch -show-current`
2. Create branch and switch to it: `git switch -c readiness/add-live-health-test`
3. Start Docker stack: `docker compose up -d`
4. Check endpoint: `curl.exe http://localhost:8001/health/live`
5. Create the test file `\tests\integration\test_health.py`:

```
def test_live_health_returns_ok(client):  
    response = client.get("/health/live")  
    assert response.status_code == 200  
    assert response.json()["status"] == "okay"
```
6. Rebuild the Docker image: `docker compose build --no-cache api`
7. Run the test (inside the container): `docker compose run --rm --no-deps api pytest -q tests/integration/test_health.py`
`pytest -q tests/integration/test_health.py`

8. If the API actually returns "ok", the test should fail. Fix it by changing in the file "okay" to "ok"
9. Rebuild the Docker image: `docker compose build --no-cache api`
10. Run again the test (it will pass)
11. Run the complete test suite: `pytest -q`
12. Review and commit the change:
`git diff`
`git add tests/integration/test_health.py`
`git commit -m "Add integration test for API live-health endpoint"`
13. Delete the practice branch afterward (if practicing locally):
`git switch main`
`git branch -D readiness/add-live-health-test`
14. Otherwise push: `git push -u origin readiness/add-live-health-test` and Go to GitHub and open a pull request. Once the checks pass and the change is approved, merge it.



ACTIVITY 7: Rehearse the loop yourself (8 min)

Repeat the demo of last slide:

clean up the drill (do not push, do not tag)

ACTIVITY 8: Draft your readiness plan card (8 min)

Write individually (save as a personal note; later reuse it in submissions/week03/README.md):

- My bounded change:
- Why it is useful:
- Files I expect to inspect/change:
- Behaviour before:
- Behaviour after:
- Verification (test / curl / logs / SQL / clean clone):
- Failure or rollback plan (git switch main; branch can be discarded):
- What I will document (submissions/week03/README.md; docs/operations.md if relevant):

Repo-grounded ideas (inspiration only — choose and own your own; identical submissions across the class will be noticed):

- /health/ready → 503 + detail when DB unavailable (verify with docker compose stop db)
- GET /jobs/{id} 404 test · GET /cases?limit=0 → 422 test · unit test for shorten() in the AI service
- Add attempts to the worker's job_claimed log line; verify in logs
- Add GET /jobs?status= filter with test
- Fix docs/operations.md test instructions (dev install, Python 3.11, container fallback); verify from a clean clone
- Add one keyword (e.g. "timeout" → incident) with a unit test
- Optional source field on Case end-to-end (schema, model, migration, response, test) — only if you can verify confidently

DEMO: The failure path and debugging order

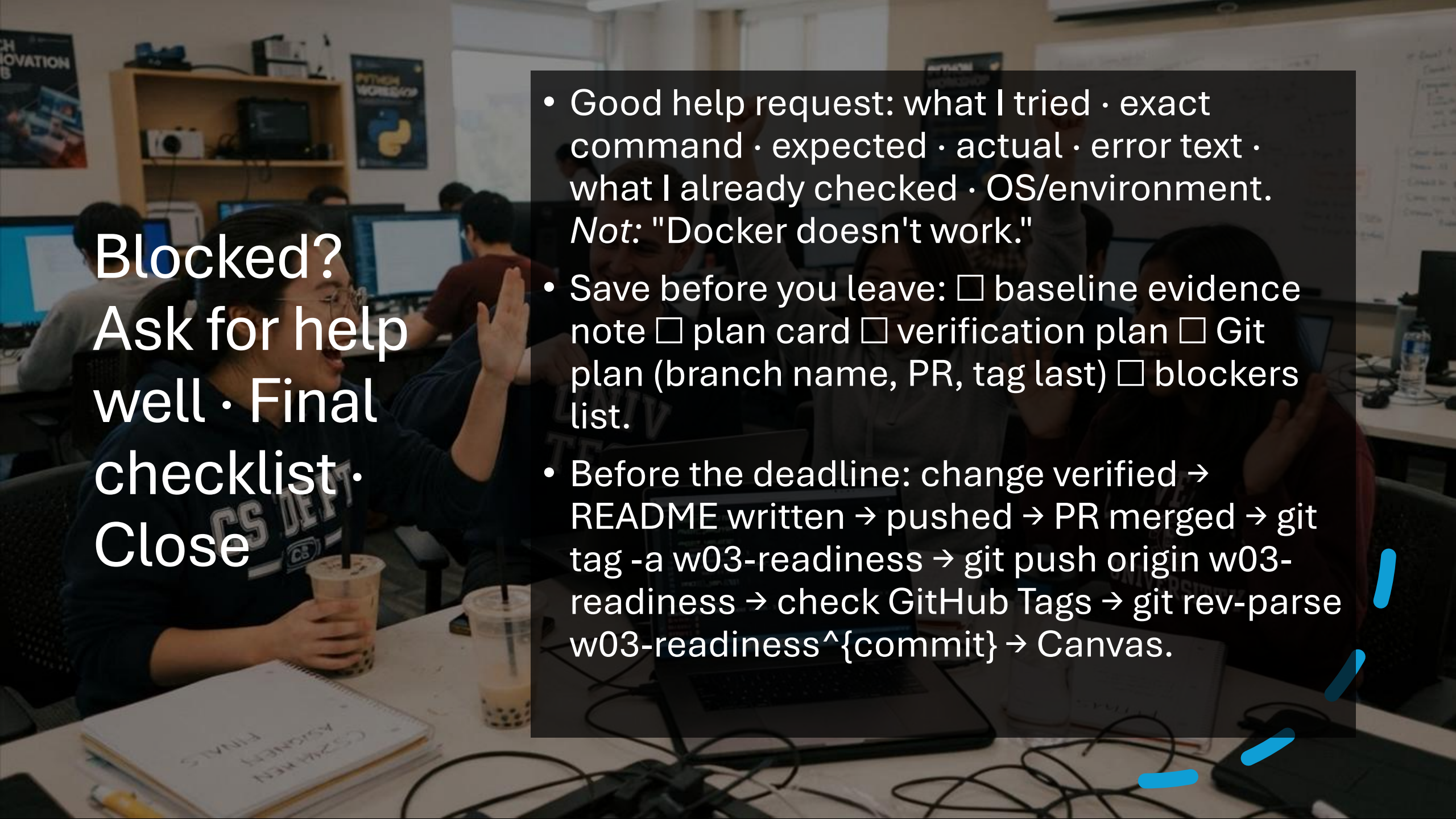
```
docker compose stop ai
# create a case via /docs → 201, status queued
docker compose logs worker --tail=10      # job_claimed → job_processing_failed
(ConnectError)
curl -s localhost:8000/jobs/<job_id>      # status "failed", attempts 1, error "..."
curl -s localhost:8000/cases/<case_id>    # status "failed"
docker compose exec db psql -U postgres -d maie6000c -c "SELECT id, status, error FROM
jobs ORDER BY id DESC LIMIT 1;"
docker compose start ai                    # note: the failed job is NOT retried
(documented limitation)
```

ACTIVITY 9: Pair check of plan cards (3 min)

Swap cards with a neighbour; each asks:

- Can you explain the current behaviour?
- What exactly is different afterwards?
- What is the smallest evidence a reviewer would accept?
- Is this one concern, or two?
- Could you defend it orally in two minutes?
If any answer is fuzzy → make the change smaller.





Blocked?
Ask for help
well · Final
checklist ·
Close

- Good help request: what I tried · exact command · expected · actual · error text · what I already checked · OS/environment. *Not: "Docker doesn't work."*
- Save before you leave: ☐ baseline evidence note ☐ plan card ☐ verification plan ☐ Git plan (branch name, PR, tag last) ☐ blockers list.
- Before the deadline: change verified → README written → pushed → PR merged → git tag -a w03-readiness → git push origin w03-readiness → check GitHub Tags → git rev-parse w03-readiness^{commit} → Canvas.